

Shomer.AI — Manual Full-Flow Testing Guide

(Debug SDK CLI)

Version: server `v0.6.0-fullflow` · **Date:** 2026-06-06 **Purpose:** Manually exercise the complete server flow — classifier → triage → context agent → alerts → audit — using the Python debug SDK, and read back everything the server persisted.

All commands run **from the repository root** (`C:\AIDevelopmentCourse\Shomer.AI`) in **PowerShell**. They call the venv Python directly, so no activation step is needed.

Two ways to test. `$0` is the **interactive console** — the easiest path: it starts the server for you, health-checks it, and gives a menu. `$$1–15` are the **manual CLI** commands behind that menu, for when you want fine control or to script individual calls.

0. Easiest path — the interactive test console

`scripts\test_console.py` is a self-contained, menu-driven tester. On launch it **starts its own isolated server**, runs a **health check**, and creates a **per-session folder** under `test-sessions\` holding that run's `audit.db` and `server.log` (separate files, so each test run is clean and auditable).

```
cd C:\AIDevelopmentCourse\Shomer.AI
server\.venv\Scripts\python.exe scripts\test_console.py
```

Useful flags: - `--port 8010` — use a different port (if `:8000` is busy). - `--attach http://localhost:8000` — drive an already-running server instead of spawning one. - `--selftest` — non-interactive smoke run (3 requests + stats), then exit. - `--no-color` — plain output.

The menu (every item carries a one-line description of its role in the full flow; press `?` for the full pipeline explanation):

Key	Function	What it exercises in the full flow
1	Classify TEXT	text → classifier → triage → (CA if borderline) → alert? → audit
2	Classify IMAGE	image → OCR → classifier → triage → (CA) → alert? → audit
3	Golden-set demo	10 curated samples through the whole pipeline
4	Conversation history	a child's stored turns (what the CA's <code>read_history</code> sees)
5	Recent classifications	last audit rows: label, triage decision, child
6	Alerts sent	dispositions: SENT vs rate_limited, severity, message
7	Audit stats	totals + triage-branch distribution
8	Replay a trace	re-run a stored input, compare the decision
9	Tail server log	last 40 lines of this session's <code>server.log</code>
t	Inspect a trace	full audit record for one <code>trace_id</code>
m	Gateway metrics	Prometheus <code>shomer_*</code> series from <code>/metrics</code>
L	Load test	concurrent burst → latency report in the session folder
i	Health / info	<code>/health</code> + <code>/model/info</code>
p	Session paths	where <code>audit.db</code> / <code>server.log</code> live
?	Help — full flow	explains each function's place in the pipeline
q	Quit	stops the spawned server (session files are kept)

After every classify, the console prints the **classifier verdict**, the **triage decision**, the **Context-Agent verdict** (if it ran), and **the alert that was sent** (label, severity, message) — or why none was. To see an alert marked **SENT**, give each offensive message a distinct `child_id` (see §13).

The rest of this guide (§§1–15) documents the same operations as standalone CLI commands.

1. Prerequisites

- **Ollama must be running** — it backs the `v1.0-standin` classifier. Verify: `powershell curl http://localhost:11434/api/tags`
 - The server virtual-env already has every dependency (`server\.venv`).
 - (Optional) Real LLM keys in `server\.env` (`OPENAI_API_KEY` / `ANTHROPIC_API_KEY`) — only needed to see *Context-Agent-confirmed* alerts. Without them the Context Agent runs as a deterministic mock (see §13).
-

2. How to run the server

Open **terminal #1** and start the server (leave it running):

```
cd C:\AIDevelopmentCourse\Shomer.AI
$env:AUDIT_DB_PATH = "server/data/audit.db"
$env:CONTEXT_AGENT_ENABLED = "true"
server\.venv\Scripts\python.exe -m uvicorn app.main:app --app-dir server --port 8000
```

You should see a `server_ready` log line listing `classifier`, `triage`, `notifier`, `audit=SqliteAuditStore`. Stop the server later with `Ctrl+C`.

Open **terminal #2** for all the CLI commands below. (Optional) point every command at the server once:

```
cd C:\AIDevelopmentCourse\Shomer.AI
$env:SHOMER_SERVER = "http://localhost:8000"
```

Networking note: the server listens on `:8000`. Android emulator reaches it at `http://10.0.2.2:8000/`; a physical phone uses the PC LAN IP plus a one-time firewall rule (see `CLAUDE.md`).

3. Liveness check

```
server\.venv\Scripts\python.exe scripts\dev_client.py health
server\.venv\Scripts\python.exe scripts\dev_client.py info
```

Expect: `status ok`, `ollama_reachable true`, model `v1.0-standin`, and the five labels `abusive`, `hate`, `violence`, `pornographic`, `non_offensive`.

4. Golden-set demo (fastest end-to-end sanity check)

```
server\.venv\Scripts\python.exe scripts\dev_client.py demo
```

Expect: 10/10 requests return 200, each matching its expected category (clean greetings + one example of each offensive label).

5. Exercise each triage branch (`classify`)

Each call prints the `trace_id` it used — note it for §7 (inspection). The `--child-id` flag persists a conversation turn and gives that child its own alert rate-limit bucket.

```
# SILENT – clean text
server\.venv\Scripts\python.exe scripts\dev_client.py classify "שלום אמא, אני בדרך הביתה" --child-id alon

# ALERT_DIRECT – abusive, high confidence -> a SENT alert (fresh child bucket)
server\.venv\Scripts\python.exe scripts\dev_client.py classify "אתה טמבל מטומטם, כולם שונאים אותך" --child-id alon

# ALERT_DIRECT – pornographic (always-alert override)
server\.venv\Scripts\python.exe scripts\dev_client.py classify "תשלחי לי תמונות עירום שלך עכשיו" --child-id alon

# ESCALATE_TO_CA – violence always escalates to the Context Agent
server\.venv\Scripts\python.exe scripts\dev_client.py classify "אני אשבור לך את העצמות אחרי בית הספר" --child-id alon
```

Expect: `t-clean` → `non_offensive` ; `t-abuse` → `abusive` (offensive); `t-porn` → `pornographic` ; `t-violence` → `violence` . The triage *decision* and whether an alert fired are confirmed in §7.

6. Conversation history (the `child_id` path)

Send several messages under the **same** `--child-id` . Each is stored as a conversation turn that the Context Agent's `read_history` tool can read back:

```
server\.venv\Scripts\python.exe scripts\dev_client.py classify "מה נשמע היום?" --child-id alon --trace-id alon
server\.venv\Scripts\python.exe scripts\dev_client.py classify "מתי ארוחת ערב מוכנה?" --child-id alon --trace-id alon
```

7. Inspect what the server persisted (`inspect_audit.py`)

```
$DB = "server/data/audit.db"

# Totals + triage/label/alert distributions
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB stats

# Most recent classifications
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB recent --limit 15
```

```
# Full picture for ONE request (classification + agent trace + alert)
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB trace t-abuse
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB trace t-violence

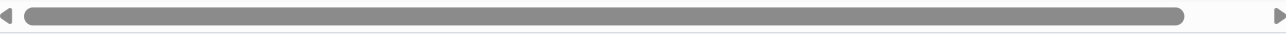
# A child's conversation turns + their classifications
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB child alon

# Alert dispositions (sent / rate_limited)
server\.venv\Scripts\python.exe scripts\inspect_audit.py --db $DB alerts
```

Expect: - `trace t-abuse` → `triage_decision: alert_direct` and an *Alerts* section with `fcm_status: sent`, severity `medium`. - `trace t-violence` → an *Agent traces* section (`tools_called: read_history, lookup_slang, check_age`, a Hebrew explanation). - `child alon` → the conversation turns in order plus the classifications.

8. Image path (`classify-image`)

```
server\.venv\Scripts\python.exe scripts\dev_client.py classify-image path\to\screenshot.png --child-id
```



OCR extracts the Hebrew text, then it runs the same pipeline as `/classify`. An empty or unreadable image returns `REVIEW_NEEDED` and fires no alert.

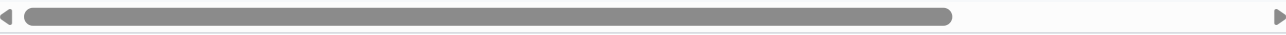
9. Replay a recorded request (regression aid)

```
server\.venv\Scripts\python.exe scripts\dev_client.py replay t-abuse --db server/data/audit.db
```

Reads the stored input for that trace, re-issues it, and prints the stored-vs-fresh decision (exit code 2 if the label changed).

10. Load / concurrency test → Markdown report

```
server\.venv\Scripts\python.exe scripts\load_test.py --concurrency 4 --repeat 2 --db server/data/audit
```



Open `docs\loadtest_report.md` for p50/p95/p99 latency, error rate, and a triage-branch histogram.

11. Gateway metrics

```
curl http://localhost:8000/metrics | Select-String "shomer_"
```

Shows the Prometheus series the Gatekeeper exports (request durations, rate-limit and payload counters, gateway overhead).

12. Expected results at a glance

You send	Triage branch	Alert fired?
clean text	<code>silent</code>	no
<code>abusive</code> / <code>hate</code> (high confidence)	<code>alert_direct</code>	yes — sent (with a fresh <code>--child-id</code>)
<code>pornographic</code>	<code>alert_direct</code> (always-alert override)	yes — sent
<code>violence</code>	<code>escalate_to_ca</code> → Context Agent	with the mock CA it ends <code>silent</code> → no alert (see §13)
garbage / unreadable image	<code>review_needed</code>	no

13. Two behaviors to keep in mind (both by design)

1. **Mock Context Agent.** With no `OPENAI_API_KEY` / `ANTHROPIC_API_KEY` in `server\.env` , the Context Agent uses a deterministic mock that resolves every escalated/borderline case (including all `violence`) to "not a threat" → final decision `silent` . The path still runs and records an agent trace — it just does not *alert*. Add real keys to see CA-confirmed alerts.
 2. **Anti-storm rate limiting.** Messages sent **without** `--child-id` share a single `"unknown"` rate-limit bucket, so only 3 alerts per minute are sent and the rest are recorded as `rate_limited` . Use a **distinct** `--child-id` per offensive message to see `sent` . Separately, the Gatekeeper caps **100 requests/minute per IP** — beyond that you get HTTP `429` .
-

14. Reset between runs

```
# Stop the server (Ctrl+C in terminal #1), then:  
Remove-Item server/data/audit.db* -ErrorAction SilentlyContinue  
# Restart the server (§2) for a clean audit database.
```

15. Command reference

Tool	Subcommand	Purpose
dev_client.py	health / info	server liveness + model metadata
	classify <text> [--child-id --message-id --trace-id]	classify Hebrew text
	classify-image <path> [--child-id --trace-id]	OCR + classify an image
	demo [--json]	run the curated golden set
	replay <trace_id> [--db]	re-run a recorded input
inspect_audit.py	stats / recent / trace <id> / child <id> / alerts	read-only audit-DB inspector
load_test.py	--concurrency --repeat --db --out	concurrency + latency report

Common options: `--server <url>` (or `$env:SHOMER_SERVER`), `--timeout <sec>`, `--db <path>` (default `server/data/audit.db`).